Suppose you add a new event called `QuarterReached` to the sharded container to indicate that it has reached a quarter of its capacity. Upon receiving this event, or after a certain number of events, you want to take a snapshot:

```
EventSourcedBehavior[Command, Event, State](...).snapshotWhen {
    case (state, QuarterReached(_), sequenceNumber) => true       ❶
    case (state, event, sequenceNumber)             => false      ❷
 }
 .withRetention(
     RetentionCriteria.snapshotEvery(numberOfEvents = 100,   keepNSnapshots = 2))
❸
```

❶ Snapshots each time the entity receives QuarterReached

❷ Doesn't snapshot otherwise when matching events

❸ Snapshots each 100 events, and keeps the two previous snapshots

A snapshot is created every time the message `QuarterReached` is received by the actor and after every 100 events are processed. In both cases, `keepNSnapshots` indicates that you are deleting previous snapshots. Here only the last two are available. This releases resources while keeping two previous snapshots in case something goes wrong.

By default, the snapshot feature is enabled and configured to read the last snapshot. However, if an error occurs, a `SnapshotFailed` signal is triggered, but the actor does not stop; it just logs the error.

If you know you have a corrupted snapshot or that it is incompatible with your actor—after you have written a new version—you can avoid loading it by setting the recovery to `SnapshotSelectionCriteria.none`:

```
EventSourcedBehavior[Command, Event, State](...)
  .withRecovery(Recovery.withSnapshotSelectionCriteria(SnapshotSelectionCriteria.none)
```

9.3.4 Tagging

When you save to the journal, you usually mark each event so other processes can read the journal by filtering for those marks. After all, the journal is just a table in a database. The tagging is done as follows:

```
EventSourcedBehavior[Command, Event, State](...)
  .withTagger( _ => Set("tag1", "tag2"))          ❶
```

❶ Adds these two tags no matter what event

Another typical use case for tagging is to parallelize reading by assigning a set of IDs to a tag so each reader can focus on a particular tag, just like a partition. You can use both in the container to tag specific events and parallelize: